

RETAIL SALES ANALYSIS

Azure Data Factory – End-to-End Pipeline Documentation

Medallion Architecture | Bronze → Silver → Gold

1. Project Overview

This project implements a fully automated, cloud-native retail analytics pipeline on Azure Data Factory (ADF). It ingests raw sales, customer, and product data from heterogeneous sources, transforms it through a three-layer Medallion architecture, and surfaces a unified analytical dataset in Azure SQL Database for downstream BI consumption.

The pipeline is designed around three guiding principles:

- **Reliability:** Every data source is read atomically; failures in one ingestion branch do not cascade to others.
- **Freshness:** A daily schedule trigger (GMT midnight) ensures analysts always work with current data.
- **Traceability:** Audit columns (pipeline_run_id, load_date, pipeline_name) are injected at the REST ingestion step so every row can be traced back to its pipeline execution.

1.1 Business Objective

The project answers the following analytical questions for a UK-focused retail operation:

- Which non-book products are running low on stock (ranked by stock level)?
- Who are the UK-based customers and how do they rank within the country?
- Which sales transactions represent low-volume orders (quantity below 10) and what is their revenue contribution, sorted by highest total amount?
- What is the complete sales picture when customers and products are joined to transactions for Gold-layer consumption?

2. Architecture Overview

The solution follows the Medallion Architecture pattern with three storage layers inside Azure Data Lake Storage Gen2 (ADLS Gen2):

Layer	Zone	Description
Bronze	Raw / Landing	Raw data as received from source systems. No transformation. Preserves original fidelity including all columns and data types.
Silver	Curated	Filtered, ranked, and cleansed data sets. Business logic is applied here. Three separate Silver datasets are produced (Products, Customers, Sales).
Gold	Analytical	Joined, conformed dataset written to Azure SQL Database. Ready for direct BI tool connection (Power BI, SSRS, etc.).

2.1 High-Level Data Flow

The pipeline follows a fan-out / fan-in pattern:

Three ingestion branches run in parallel (Customer CSV, Products On-Prem SQL, Sales REST API). All three must succeed before the Silver transformation begins. The Silver (Minor) data flow must complete before the Gold (Major) data flow executes.

- Branch A — Customer CSV → Bronze Parquet → Silver Customers Parquet
- Branch B — On-Premises SQL Server (Products) → Bronze Parquet → Silver Products Parquet
- Branch C — REST API (Sales, paginated) → Bronze Parquet → Silver Sales Parquet
- Merge — Silver Customers + Silver Products + Silver Sales → Gold Azure SQL Table

2.2 Azure Services Used

Service	Role	Resource Name	Notes
Azure Data Factory	Orchestration engine	RetailSale_GB_Pipeline01	Hosts pipeline, data flows, triggers
ADLS Gen2	Bronze & Silver storage	retailsalesgb	Hierarchical namespace enabled
Azure SQL Database	Gold layer sink	retail-gb / RetailDB	Encrypted; credentials via Key Vault
Azure Key Vault	Secret management	DataSecurePro	Stores AzSQLPass secret
Self-Hosted IR	On-prem connectivity	IR-Retail-GB	Bridges on-prem SQL Server

On-Prem SQL Server	Products source	Server / Database	Connected via Self-Hosted IR
GitHub (raw.githubusercontent.com)	Sales REST API & manifest	Username/RetailSales	Anonymous HTTP; paginated JSON

3. Repository Structure

```

retail-sales-analysis/
  adf-pipeline/
    RetailSale_GB_Pipeline01.json      # Main ADF pipeline definition
  data-flow/
    RetailSale_DF.json                # Mapping Data Flow - Bronze to Silver
    RetailSale_DF2.json               # Mapping Data Flow - Silver to Gold
  dataset/
    ds_CSV_Customers.json             # Source: Customer CSV on ADLS
    ds_Parq_Customers.json            # Bronze: Customer Parquet
    ds_Parq_Products.json             # Bronze: Products Parquet
    ds_Parq_Sales.json                # Bronze: Sales Parquet
    ds_Parq_Silver.json               # Silver: Products (ranked)
    ds_Parq_Silver_Customers.json     # Silver: Customers (ranked)
    ds_Parq_Silver_Sales.json         # Silver: Sales (filtered)
    ds_Silver_DF2_Customer.json       # DF2 Silver Customer source
    ds_Silver_DF2_Products.json       # DF2 Silver Products source
    ds_Sales_Source.json              # REST API paginated source
    ds_Sales_Parq.json                # Bronze: Sales Parquet sink
    ds_Table_Products.json            # On-prem SQL Products table
    ds_AZSQLDBCust.json               # Gold: Azure SQL sink
    ds_manifest.json                  # Sales pagination manifest
  integration-runtime/
    IR-Retail-GB.json                 # Self-Hosted IR definition
  linked-service/
    AzureDataLakeStorage1.json        # ADLS Gen2 primary connection
    ls_AzSQL.json                     # Azure SQL Database
    ls_CSV_Customers.json              # ADLS - Customer CSV
    ls_Manifests.json                 # HTTP - Sales manifest
    ls_Sales_Sources.json              # REST - Sales pages
    ls_Table_Products.json             # On-Prem SQL (via IR)
    ls_Parq_*.json                    # ADLS linked services (Bronze/Silver)
    SQLPass.json                      # Azure Key Vault reference
  trigger/
    DailyUpdate.json                  # Schedule trigger (daily, GMT)
  sample-data/
    Customers.csv                     # 500 customer records
    Products.sql                      # Products DDL + INSERT statements
    Sales/
      manifest.json                   # Pagination manifest (10 pages)
      sales_page_1.json ... sales_page_10.json # 1,000 sales records
  architecture/
    Architecture.jpg                   # Architecture diagram

```

4. Data Sources

4.1 Customers (CSV)

Source type: Delimited text (CSV) stored in Azure Data Lake Storage Gen2.

Column	Type	Description
CustomerID	String	Unique customer identifier
CustomerName	String	Full name
Email	String	Contact email address
Phone	String	International format phone number
City	String	City of residence
Country	String	Country (e.g., UK, USA, India)

Sample size: 500 customer records across multiple countries. The Silver layer filters to UK-only customers.

4.2 Products (On-Premises SQL Server)

Source type: SQL Server table (on-premises server, database) accessed through the Self-Hosted Integration Runtime IR-Retail-GB.

Column	Type	Description
ProductID	INT (IDENTITY)	Auto-incrementing primary key
ProductName	VARCHAR(100)	Descriptive product name
Category	VARCHAR(50)	Product category (e.g., Electronics, Clothing, Books)
Price	DECIMAL(10,2)	Unit selling price
StockQuantity	INT	Current inventory count

Categories observed in sample data: Books, Clothing, Electronics, Automotive, Beauty & Health, Sports & Outdoors, Toys & Games, Home & Kitchen, Office Supplies, Garden. The Silver layer excludes the Books category.

4.3 Sales (Paginated REST API)

Source type: REST API served via GitHub raw content. The manifest file is read first to determine the total page count, then each page is fetched in parallel via a ForEach loop.

Manifest structure (manifest.json):

- totalRecords: 1,000
- totalPages: 10
- pageSize: 100 records per page
- pages[]: Array of page URLs

Column	Type	Description
SaleID	Long	Unique sale transaction identifier
CustomerID	Long	Foreign key referencing Customers
ProductID	Long	Foreign key referencing Products
SaleDate	String	Date of sale (YYYY-MM-DD format)
Quantity	Long	Units purchased in this transaction
TotalAmount	Double	Total sale value in local currency

Three audit columns are injected automatically by ADF at ingestion time:

- pipeline_run_id — ADF pipeline run GUID for lineage tracing
- load_date — UTC timestamp when the record was loaded
- pipeline_name — Name of the executing pipeline

5. Pipeline – RetailSale_GB_Pipeline01

5.1 Activity Overview

The pipeline contains 7 activities organised in a dependency graph. Three activities run in parallel at pipeline start; all must succeed before transformation begins.

Activity	Type	Depends On	Description
Customer_CSV	Copy	None (parallel start)	Copies Customer CSV from ADLS to Bronze Parquet
Products_OnPrem	Copy	None (parallel start)	Copies Products table from on-prem SQL to Bronze Parquet via Self-Hosted IR
LookupManifest	Lookup	None (parallel start)	Reads Sales pagination manifest to determine total page count
setPageArray	SetVariable	LookupManifest Succeeded	Builds an integer array [1 ... totalPages] using the range() expression
ForEach1	ForEach	setPageArray Succeeded	Iterates over page array; each iteration copies one page of Sales JSON to Bronze Parquet
Minor	ExecuteDataFlow	Customer_CSV + Products_OnPrem + ForEach1 all Succeeded	Executes RetailSale_DF: Bronze to Silver transformation
Major	ExecuteDataFlow	Minor Succeeded	Executes RetailSale_DF2: Silver to Gold join and SQL write

5.2 Parallel Ingestion Pattern

Customer_CSV, Products_OnPrem, and LookupManifest/ForEach1 have no dependencies on each other, so ADF executes them concurrently. This fan-out maximises throughput during the ingestion phase and minimises total pipeline duration.

5.3 Paginated Sales Ingestion

Sales data is delivered as a paginated JSON API with 10 pages of 100 records each. The pipeline handles pagination dynamically rather than hardcoding page counts:

- Step 1: LookupManifest reads the manifest JSON and extracts totalPages.
- Step 2: setPageArray uses the ADF expression `@range(1, activity('LookupManifest').output.firstRow.totalPages)` to create a numeric array.
- Step 3: ForEach1 iterates the array; each iteration calls the SalesData Copy activity with the current page index as a dynamic relative URL parameter.

This design means the pipeline automatically handles changes to the page count without any configuration changes.

5.4 Pipeline Variables

Variable Name	PageArray
Type	Array
Purpose	Holds the dynamically-generated array of page numbers used by ForEach1
Set By	setPageArray activity using @range() expression

5.5 Timeout & Retry Configuration

All Copy and ExecuteDataFlow activities share the same policy:

- Timeout: 12 hours (0.12:00:00)
- Retry count: 0 (no automatic retry)
- Retry interval: 30 seconds

The timeout is intentionally generous to accommodate large data volumes. Retry is disabled to prevent duplicate data writes to Parquet sinks; idempotency is handled by the pipeline trigger schedule instead.

6. Data Flows

6.1 RetailSale_DF – Bronze to Silver (Minor)

This Mapping Data Flow reads all three Bronze datasets simultaneously and produces three independent Silver datasets. Compute: 8 cores, General purpose.

6.1.1 Products Stream → Silver Products

Step	Transformation	Logic
1	Source: Products	Reads Bronze Products Parquet (ProductID, ProductName, Category, Price, StockQuantity)
2	Filter: NonBooks	Category != 'Books' — removes all book products from downstream
3	Sort: sort1	asc(StockQuantity) — orders by stock ascending to surface low-stock items
4	Rank: rank2	desc(StockQuantity) — assigns StockLevel rank (1 = highest stock). Note: rank is computed on the ascending-sorted stream, so rank 1 = lowest stock
5	Sink: sink1	Writes to Silver Products Parquet in ADLS Gen2

6.1.2 Customers Stream → Silver Customers

Step	Transformation	Logic
1	Source: Customers	Reads Bronze Customers Parquet (CustomerID, CustomerName, Email, Phone, City, Country)
2	Filter: filter2	Country === 'UK' — retains only United Kingdom customers
3	Rank: rank1	asc(Country) — assigns CountryRank within the UK group
4	Sink: sink2	Writes to Silver Customers Parquet in ADLS Gen2

6.1.3 Sales Stream → Silver Sales

Step	Transformation	Logic
1	Source: Sales	Reads Bronze Sales Parquet (SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount) plus the three injected audit columns
2	Filter: filter3	Quantity < 10 — isolates low-quantity (single/small batch) transactions

3	Sort: sort2	desc(TotalAmount) — highest-revenue low-quantity transactions first
4	Sink: sink3	Writes to Silver Sales Parquet in ADLS Gen2

6.2 RetailSale_DF2 – Silver to Gold (Major)

This Mapping Data Flow reads the three Silver datasets and produces a single conformed Gold table in Azure SQL Database. Compute: 8 cores, General purpose.

Step	Transformation	Logic
1	Source: source1	Reads Silver Customers Parquet (includes CountryRank)
2	Source: source2	Reads Silver Products Parquet (includes StockLevel)
3	Source: source3	Reads Silver Sales Parquet (filtered, sorted)
4	Join: ProductSales	RIGHT JOIN source2 (Products) ON source2.ProductID == source3.ProductID. Retains all Silver Sales rows; attaches product attributes where available
5	Join: join1	LEFT JOIN on source3.CustomerID == source1.CustomerID. Attaches customer attributes to the product-sales joined stream
6	Sink: GoldLayer	INSERT into Azure SQL Database table (ds_AZSQLDBCust). Error handling: stop on first error. No deletes or updates; append-only.

The join strategy (right join for products, left join for customers) ensures all sales transactions appear in the Gold layer even if a product or customer record is missing, preserving full sales history at the cost of nullable dimension columns.

7. Linked Services

Linked Services define the connection parameters for each external system. Credentials are never stored in plain text; they are either encrypted by ADF or referenced from Azure Key Vault.

Linked Service	Type	Endpoint	Auth Method
AzureDataLakeStorage1	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_CSV_Customers	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Par_Customers	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Parq_Products	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Parq_Sales	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Parq_Silver_Product	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Parq_Silver_Sales	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Sales_Parq	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Silver_DF2_Customer	AzureBlobFS	retailsalesgb.dfs.core.windows.net	ADF Encrypted Credential
Is_Manifests	HttpServer	raw.githubusercontent.com (manifest.json)	Anonymous
Is_Sales_Sources	RestService	raw.githubusercontent.com/SreeLekshmi00/RetailSales/main/Sales/	Anonymous
Is_Table_Products	SqlServer	ServerName / DBName (on-prem, via IR)	SQL Auth (encrypted credential)
Is_AzSQL	AzureSqlDatabase	retail-gb.database.windows.net / RetailDB	SQL Auth + Azure Key Vault secret

SQLPass	AzureKeyVault	DataSecureProject1.vault.azure.net	Managed Identity / ADF Service Principal
---------	---------------	------------------------------------	--

7.1 Secret Management

The Azure SQL Database password is stored as a secret named AzSQLPass in Azure Key Vault (vault: DataSecureProject1). The linked service ls_AzSQL references this secret via the SQLPass Key Vault linked service. No passwords appear anywhere in the ADF JSON configuration files.

8. Datasets

8.1 Bronze Layer Datasets

Dataset	Format	Description
ds_CSV_Customers	CSV (DelimitedText)	Source customer file on ADLS Gen2
ds_Parq_Customers	Parquet	Bronze customer data landing zone
ds_Table_Products	SQL Server Table	On-prem Products table source
ds_Parq_Products	Parquet	Bronze products landing zone on ADLS
ds_Sales_Source	REST / JSON	Parameterised REST source; relativeURL passed at runtime by ForEach
ds_manifest	JSON (HTTP)	Manifest file read by LookupManifest activity
ds_Sales_Parq	Parquet	Bronze sales landing zone (FlattenHierarchy write)

8.2 Silver Layer Datasets

Dataset	Format	Description
ds_Parq_Silver	Parquet	Silver Products output (NonBooks, ranked by stock level)
ds_Parq_Silver_Customers	Parquet	Silver Customers output (UK only, country-ranked)
ds_Parq_Silver_Sales	Parquet	Silver Sales output (Quantity < 10, sorted by TotalAmount desc)
ds_Silver_DF2_Customer	Parquet	Silver Customers as read by RetailSale_DF2
ds_Silver_DF2_Products	Parquet	Silver Products as read by RetailSale_DF2

8.3 Gold Layer Dataset

Dataset	Format	Description
ds_AZSQLDBCust	Azure SQL Table	Gold output: joined sales, products, and customer data written to RetailDB on retail-gb.database.windows.net

9. Integration Runtime

9.1 IR-Retail-GB – Self-Hosted

The project uses a Self-Hosted Integration Runtime named IR-Retail-GB to bridge the on-premises SQL Server to Azure Data Factory.

IR Name	IR-Retail-GB
Type	Self-Hosted
Purpose	Enables ADF to reach the on-premises SQL Server where the Products table resides
Used By	Is_Table_Products linked service
Host Server	ServerName(on-premises network)
Database	YourDB

Self-Hosted IR must be installed and running on a machine with network access to ServerName. The ADF-managed cloud IR cannot reach on-premises resources directly.

9.2 Cloud IR

All Azure-to-Azure connections (ADLS Gen2, Azure SQL Database, REST/HTTP endpoints) use the default Azure AutoResolveIntegrationRuntime. No additional cloud IR configuration is required.

10. Trigger – DailyUpdate

Trigger Name	DailyUpdate
Type	ScheduleTrigger
Frequency	Daily
Interval	1 (every day)
Start Time	2026-05-01T00:00:00
Time Zone	GMT Standard Time
Pipeline	RetailSale_GB_Pipeline01
Current State	Stopped (activate before use)

The trigger is deployed in Stopped state. It must be activated in the ADF Studio or via the ADF REST API before it will fire. Once active, it will execute the full pipeline once per day at midnight GMT.

11. Sample Data

The sample-data/ directory contains representative data for local testing and pipeline validation without connecting to live source systems.

11.1 Customers.csv

- 500 records
- Multi-country: UK, USA, India, and others
- Columns: CustomerID, CustomerName, Email, Phone, City, Country
- Use: Upload to the ADLS container path configured in ds_CSV_Customers to simulate the customer source

11.2 Products.sql

- Contains the CREATE TABLE DDL for the Products table
- Followed by an INSERT ... VALUES block covering multiple categories including Books, Electronics, Clothing, Automotive, Beauty & Health, Sports & Outdoors, Toys & Games, Home & Kitchen, Office Supplies, and Garden
- Use: Execute against the database on the server to seed the on-prem source

11.3 Sales JSON Pages

- 1,000 total sales records across 10 JSON files (100 records each)
- Date range: from 2023-01-01
- Each record: SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount
- Hosted at: <https://raw.githubusercontent.com/SreeLekshmi00/RetailSales/main/Sales/>
- manifest.json defines the pagination metadata (totalRecords, totalPages, pageSize, page URLs)

12. Deployment Guide

12.1 Prerequisites

- Azure subscription with Contributor access
- Azure Data Factory instance (any region)
- Azure Data Lake Storage Gen2 account (retailsalesgb or equivalent)
- Azure SQL Database (retail-gb server, RetailDB database) with a table configured to receive the Gold output
- Azure Key Vault with a secret named AzSQLPass containing the Azure SQL password
- On-premises SQL Server with database and Self-Hosted IR installed and registered

12.2 Deployment Steps

1. Clone this repository to your local machine.
2. Install the Self-Hosted Integration Runtime on the on-prem server and register it with your ADF instance. Update IR-Retail-GB.json if the IR name differs.
3. Update all linked service JSON files: replace the ADLS endpoint URL, SQL Server name, Azure SQL server/database, and Key Vault URL with your actual resource names.
4. Import linked services into ADF via ADF Studio (Manage > Linked Services > New > Import from JSON) or use the ADF REST API.
5. Import datasets into ADF (Author > Datasets > New > Import from JSON for each file in the dataset/directory).
6. Import data flows: Author > Data Flows > New > Import from JSON for RetailSale_DF.json and RetailSale_DF2.json.
7. Import the pipeline: Author > Pipelines > New > Import from JSON using RetailSale_GB_Pipeline01.json.
8. Import the trigger: Manage > Triggers > New > Import from JSON using DailyUpdate.json. Activate the trigger once ready.
9. Seed the on-premises SQL Server using Products.sql. Upload Customers.csv to the configured ADLS container path.
10. Trigger a manual pipeline run from ADF Studio and monitor activity execution in the Monitor tab.

12.3 Configuration Reference

The following values must be updated for your environment before deployment:

Parameter	File(s)	Description
ADLS endpoint URL	All Is_*.json (AzureBlobFS type)	Replace retailsalesgb.dfs.core.windows.net with your storage account DFS endpoint
SQL Server hostname	Is_Table_Products.json	Replace ServerName with your on-prem SQL Server name or IP
SQL Database name	Is_Table_Products.json	Replace DBName with your on-prem database name

Azure SQL server	Is_AzSQL.json	Replace retail-gb.database.windows.net with your Azure SQL server FQDN
Azure SQL database	Is_AzSQL.json	Replace RetailDB with your database name
Key Vault URL	SQLPass.json	Replace DataSecureProject1.vault.azure.net with your Key Vault URL
SQL username	Is_AzSQL.json, Is_Table_Products.json	Update username fields (Niranjan / Niran) to your SQL login
IR name	Is_Table_Products.json	Replace IR-Retail-GB if your Self-Hosted IR has a different name
Sales manifest URL	Is_Manifests.json	Update if hosting sample data in a different GitHub repo
Sales REST base URL	Is_Sales_Sources.json	Update to match your Sales JSON hosting location

13. Security Considerations

13.1 Credential Storage

- All Azure Data Lake Storage credentials are stored as ADF Encrypted Credentials and are never exposed in the JSON configuration files.
- The Azure SQL Database password is stored exclusively in Azure Key Vault (secret: AzSQLPass) and is referenced dynamically at runtime. It is not present in any JSON file in this repository.
- On-premises SQL Server credentials are stored as ADF encrypted credentials associated with the Self-Hosted IR and are not stored locally on disk.

13.2 Network Security

- Azure SQL Database is configured with mandatory TLS encryption (encrypt: mandatory).
- The Self-Hosted IR provides a secure outbound tunnel from the on-premises network to ADF without requiring any inbound firewall ports.
- The REST API and manifest endpoints use HTTPS with server certificate validation enabled.

13.3 What This Repository Does NOT Contain

- No passwords, connection strings, or storage account keys.
- No ADF encryption keys or credential IDs that are redeemable outside the originating ADF instance.
- The encryptedCredential fields in linked service JSON files are ciphertext encrypted by the originating ADF instance and are not portable; they must be re-encrypted when imported into a different ADF instance.

Before committing any changes to this repository, verify that no plaintext secrets have been introduced into linked service or dataset JSON files.

14. Monitoring & Observability

14.1 ADF Monitor Tab

All pipeline runs, activity runs, and trigger runs are visible in the ADF Studio Monitor tab. Key metrics to observe:

- Pipeline run status: Succeeded / Failed / In Progress
- Activity duration: Particularly the ForEach1 loop and both ExecuteDataFlow activities
- Data read / written metrics on each Copy activity
- Data flow execution statistics (rows processed, partitions, data previews)

14.2 Trace Level

Both data flows (RetailSale_DF and RetailSale_DF2) are configured with traceLevel: Fine. This enables detailed per-partition row count statistics in the ADF monitoring pane, which is useful for diagnosing filter selectivity (e.g., confirming how many UK customers remain after the filter step).

14.3 Audit Columns

The three audit columns injected during Sales ingestion (pipeline_run_id, load_date, pipeline_name) flow through to the Gold layer. These allow post-hoc queries against the Gold table to identify exactly which pipeline run produced each row, enabling incremental debugging and data lineage tracking.

15. Known Limitations & Design Decisions

15.1 Current Limitations

- No incremental / delta load: Each pipeline run ingests the full dataset. For large production datasets, an incremental strategy (watermark or change data capture) should be implemented.
- Retry disabled: Copy and Data Flow activities have retry count set to 0. Transient network failures will cause the pipeline to fail immediately. Consider enabling retry (1-3 attempts) for production.
- Gold layer is append-only: The GoldLayer sink is insert-only (no upsert or delete). Repeated runs will append duplicate rows. A pre-run DELETE or TRUNCATE step, or an upsert strategy, is needed for idempotent Gold writes.
- Sales data is static: The current REST source points to a fixed GitHub dataset. In production this would be replaced with a live API or database source.

15.2 Design Decisions

- Parquet as intermediate format: Parquet is chosen for Bronze and Silver storage for its columnar compression, schema enforcement, and native compatibility with ADF Mapping Data Flows and Spark-based engines.
- Manifest-driven pagination: Reading page count from a manifest file rather than hardcoding 10 pages makes the ingestion robust to future changes in data volume without pipeline modifications.
- Self-Hosted IR for on-prem: Avoids VPN or ExpressRoute requirements by using an outbound-only IR agent installed on the on-prem network.
- Key Vault for secrets: Centralises secret rotation; updating the AzSQLPass secret in Key Vault immediately takes effect on the next pipeline run without any ADF changes.

16. Glossary

Term	Definition	Context
ADF	Azure Data Factory	Microsoft cloud ETL/ELT orchestration service
ADLS Gen2	Azure Data Lake Storage Generation 2	Hierarchical namespace object storage used for Bronze and Silver layers
Bronze Layer	Raw landing zone	Unmodified copy of source data in Parquet format
Silver Layer	Curated / transformed zone	Filtered, ranked, and cleaned datasets ready for joining
Gold Layer	Analytical / serving zone	Joined, conformed data in Azure SQL Database for BI tools
Medallion Architecture	Bronze / Silver / Gold layered data lake pattern	Industry-standard pattern for incremental data quality
Mapping Data Flow	ADF visual ETL designer with Spark execution	Used to implement Bronze-to-Silver and Silver-to-Gold transformations
Self-Hosted IR	Integration Runtime agent installed on-premises	Bridges on-prem SQL Server to ADF without VPN
Parquet	Columnar binary file format	Intermediate storage format for Bronze and Silver layers
Manifest	JSON file describing paginated dataset structure	Read by LookupManifest to determine Sales page count
ForEach	ADF activity that iterates over an array	Used to loop over Sales pages and copy each one
pipeline_run_id	ADF system variable: unique GUID per pipeline execution	Injected as audit column during Sales ingestion
Key Vault	Azure secret store	Holds the Azure SQL password (secret: AzSQLPass)
traceLevel: Fine	Data Flow execution verbosity setting	Enables detailed row-count stats per transformation step in Monitor

End of Documentation